



PDF Download
3310273.3323053.pdf
09 February 2026
Total Citations: 31
Total Downloads: 593

 Latest updates: <https://dl.acm.org/doi/10.1145/3310273.3323053>

RESEARCH-ARTICLE

Quantum computing simulator on a heterogenous HPC system

JUN DOI, IBM Research - Tokyo, Tokyo, Japan

HITOMI TAKAHASHI, IBM Research - Tokyo, Tokyo, Japan

RUDY RAYMOND, IBM Research - Tokyo, Tokyo, Japan

TAKASHI IMAMICHI, IBM Research - Tokyo, Tokyo, Japan

HIROSHI HORII, IBM Research - Tokyo, Tokyo, Japan

Open Access Support provided by:

IBM Research - Tokyo

Published: 30 April 2019

[Citation in BibTeX format](#)

CF '19: Computing Frontiers Conference
April 30 - May 2, 2019
Alghero, Italy

Conference Sponsors:
[SIGMICRO](#)

Quantum Computing Simulator on a Heterogeneous HPC System

Jun Doi, Hitomi Takahashi, Rudy Raymond, Takashi Imamichi, Hiroshi Horii

{doichan, hitomi, rudyhar, imamichi, horii}@jp.ibm.com

IBM Research – Tokyo

Tokyo, Japan

ABSTRACT

Quantum computing simulation on a classical computer is difficult due to the exponential runtime and memory overhead. Previous work addresses the difficulty by utilizing multiple Graphical Processing Units (GPUs) and multi-node computers. GPUs are efficient for handling runtime issues but have limited total accessible memory space. Meanwhile, the memory of a multi-node computer can be scaled to the petabytes order, but its bandwidth for access from host computers (CPUs) is narrow. To simultaneously accelerate simulation and enlarge the total memory space, we propose a heterogeneous parallelization approach by combining GPUs and CPUs. Our simulator allocates memory to the GPUs first, and then to the CPUs. It thus accelerates simulation by using the full capabilities of the GPUs if memory for the simulation fits in the GPUs on a cluster. Allocating memory to the CPUs reduces benefits of the GPUs but enlarges the capacity of qubits in the simulation. In such case, it can exploit the memory of the GPUs to add one more qubit in the simulation if the size of memory in a node is the power of two (such as 512GB). We show empirical performance evaluations of our simulator in a distributed environment of POWER9.

CCS CONCEPTS

• **Computing methodologies** → **Massively parallel and high-performance simulations**; • **Software and its engineering** → **Massively parallel systems**.

KEYWORDS

Quantum Computer, Quantum Computing, Simulation, GPU, CUDA, Heterogeneous Parallelization

ACM Reference Format:

Jun Doi, Hitomi Takahashi, Rudy Raymond, Takashi Imamichi, Hiroshi Horii. 2019. Quantum Computing Simulator on a Heterogeneous HPC System. In *Proceedings of the 16th conference on Computing Frontiers (CF '19)*, April 30-May 2, 2019, Alghero, Italy. ACM, New York, NY, USA, Article 4, 9 pages. <https://doi.org/10.1145/3310273.3323053>

1 INTRODUCTION

Quantum computers (QCs) with a small number of quantum bits (qubits), although noisy and limited in operations, are now available

for running basic quantum algorithms [1, 13, 15]. Some of QCs work as a service [17], but their use are limited. Classical simulation of QC is vital for the study and development of QC algorithms. In the simulation, the state of n -qubit QCs, which is a complex vector of probability amplitudes with length 2^n , requires at least 16×2^n bytes of memory, as each element of the complex vector needs (by default) eight bytes for each real and imaginary part. The running time of the simulation for transforming the complex vector is proportional to 2^n , which is a severe limitation. Though specialized simulators can simulate specific types of quantum circuits with fewer resources [2, 4, 16], a universal quantum computing simulator, such as, [5, 17, 20], cannot avoid the exponential barriers to simulate general circuits, and thus requires enhanced performance.

GPUs are effective accelerators for QC simulation [10, 12, 14, 22] due to their computational and memory-bandwidth characteristics but have limitations on the total memory space accessible. For example, the most recent GPU card has only 32 GB of local memory [8], and one card can be used to simulate QCs with 30 or less qubits in practice. Theoretically, a 30-qubit QC requires only 16 GB of memory (16×2^{30} bytes), but running a GPU program also requires a certain amount of memory, so the states cannot occupy all 32 GB. Some simulators increase available memory by using multiple GPUs on a single node [22] and on multiple nodes [14], but large-scale simulation needs a large number of GPUs (for example, more than 1024 GPUs are needed to allocate 16 terabytes for a 40-qubit QC simulation). Though distributed CPU memory enables such large-scale simulation [11, 12, 19], GPU acceleration is unavailable in existing simulators.

We propose a heterogeneous approach to overcoming the GPUs and CPUs limitation. Multiple nodes and GPUs are combined to enlarge the total memory space accessible for a QC simulator while accelerating QC simulation with the GPUs as much as possible. This heterogeneous approach enables a smooth means of combining the memories of GPUs with those of CPUs while balancing their computational power. If all the probability amplitudes fit into the GPU memory, the simulation running time is GPU-dominated and much shorter than on a CPU alone. Even if the probability amplitudes do not fit into the GPU memory and must be partially allocated to the CPU memory, performance is still substantially improved.

We partition the probability amplitudes of n -qubit QCs. Most distributed simulators use a simple strategy to partition and update the 2^n amplitudes in multiple nodes: the amplitudes are divided equally among 2^m nodes [11, 12, 19]. We extend this strategy to the use of multiple GPU memories; the probability amplitudes in each machine are divided into 2^l chunks, where each chunk has $2 \times 2^{n-m-l}$ amplitudes, and all the chunks are equally allocated among 2^m nodes. Here, $16 \times 2^{n-m-l}$ is configured to be lower than the memory size of a GPU. Each node allocates the chunks to the GPU memory first, then to the CPU memory.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

CF '19, April 30-May 2, 2019, Alghero, Italy

© 2019 Association for Computing Machinery.

ACM ISBN 978-1-4503-6685-4/19/05.

<https://doi.org/10.1145/3310273.3323053>

A technical challenge is to maximize GPU acceleration for updating the probability amplitudes. In existing approaches that use multiple GPUs [14, 22], the probability amplitudes are repartitioned in GPU memory to avoid communication when updating them in a kernel. During this repartitioning, the GPUs are idle. The CPU memory involved in repartitioning is used only as a temporary space, and does not contribute to the scale of the simulation. A GPU communicates the probability amplitudes to the other GPUs while updating them. The patterns of communication and the updates vary depending on the gates to be simulated.

This paper makes three key contributions:

- A novel method is presented for partitioning the probability amplitudes to simulate quantum computing while maximizing their allocation in GPU memory.
- Novel techniques are presented for efficiently updating the partitioned probability amplitudes stored in a distributed environment of GPU and CPU,
- The performances of the proposed method are empirically evaluated on 32 POWER9 nodes with 128 GPUs.

Section 2 gives an overview of QC simulation. In Section 3, we describe how we allocate probability amplitudes on distributed parallel computers, and in Section 4 we present our heterogeneous approach. In Section 5, we describe the evaluation of our heterogeneous parallel QC simulator on an OpenPOWER cluster. We conclude in Section 6 with a brief summary and mention of future work.

2 QUANTUM COMPUTING SIMULATION

In this section, we first briefly overview QC and its simulation, and then summarize existing methods that perform simulation in parallel. In our work, we focused on simulating universal quantum computers based on quantum circuits consisting of one-qubit rotation gates and two-qubit CNOT gates. These gates are universal; i.e., any quantum circuit (for realizing some quantum algorithm) can be constructed from the CNOT and one-qubit rotation gates.

2.1 Overview

Quantum circuits apply quantum gates to qubits for running the programs of QCs. A qubit has two basis states, $|0\rangle = \begin{pmatrix} 1 \\ 0 \end{pmatrix}$ and $|1\rangle = \begin{pmatrix} 0 \\ 1 \end{pmatrix}$. While a classical bit stores 0 or 1 exclusively, a qubit allows superposition of the two basis states as $|\psi\rangle = a_0|0\rangle + a_1|1\rangle$, where probability amplitudes a_0 and a_1 are complex numbers satisfying $|a_0|^2 + |a_1|^2 = 1$. When a qubit is measured, an outcome of 0 or 1 is obtained with probability $|a_0|^2$ or $|a_1|^2$, respectively. This means two complex numbers for tracking the quantum state of a one-qubit register must be stored in the simulation.

The quantum state of an n -qubit register is a linear superposition of 2^n basis states, each of the form of the tensor product of n qubits. For example, the basis state of digit '2' (or 10 in binary) can be represented as $|2\rangle = |1\rangle \otimes |0\rangle = |10\rangle = (0, 0, 1, 0)^T$, where T denotes the transpose of a vector. Thus, the quantum state of the n -qubit register can be written as $|\psi\rangle = \sum_{i=0}^{2^n-1} a_i|i\rangle$. Note that each state $|i\rangle$ has its own probability amplitude a_i in a complex number. Overall, simulating quantum circuits requires that these 2^n complex numbers be stored to enable tracking of the evolution

of the quantum state of an n -qubit register. We call these complex numbers 'qreg.'

Quantum gates transform the quantum state of the n -qubit register by rotating its complex vector. We focus on the gate set defined by the OpenQASM specification [9]. The set consists of an arbitrary single-qubit (rotation) gate (a u3 gate) and a two-qubit gate (controlled-not or CNOT or CX). The u3 gates are rotations of the 1-qubit state and are mathematically defined as

$$u3(\theta, \psi, \lambda) = \begin{pmatrix} \cos(\theta/2) & -e^{i\lambda} \sin(\theta/2) \\ -e^{i\psi} \sin(\theta/2) & e^{i\psi+\lambda} \cos(\theta/2) \end{pmatrix}. \quad (1)$$

When applied to the k -th qubit of $\{q_n \cdots q_1\}$, a u3 gate transforms the probability amplitudes of the basis states

$|x_n \cdots x_{k+1} 0 x_{k-1} \cdots x_1\rangle$ and $|x_n \cdots x_{k+1} 1 x_{k-1} \cdots x_1\rangle$ for each x_i in $\{0, 1\}$ by linear transformation of their previous probability amplitudes. This implies that applying a single-qubit u3 gate can change all 2^n complex numbers stored for tracking the evolution of the quantum state. However, the changes can be computed in parallel, as each of the new probability amplitudes depends on its own value and on the value of a probability amplitude whose basis state is different at the k -th location.

Listing 1 shows the pseudocode for a u3 gate used to transform the state of a q-qubit in qreg with theta, phi, and lambda. Elements at k and $(k + (1 \ll q))$ in qreg are updated together, and each element is updated once.

Listing 1: Pseudocode for the u3 gate used to transform the state of q-qubit in qreg

```
1 void u3(int q, double theta, double phi, double lambda) {
2   complex I(0., 1.);
3   complex u00 = cos(theta/2.);
4   complex u01 = -exp(I*lambda)*sin(theta/2.);
5   complex u10 = exp(I*phi)*sin(theta/2.);
6   complex u11 = exp(I*(phi+lambda))*cos(theta/2.);
7   #pragma omp parallel for collapse(2)
8   for (int i = 0; i < (1<<n); i += (1<<(q+1))) {
9     for (int j = 0; j < (1<<q); j++) {
10      auto pos = i|j;
11      auto q0 = qreg[pos];
12      auto q1 = qreg[pos|(1<<q)];
13      qreg[pos] = u00*q0+u01*q1;
14      qreg[pos|(1<<q)] = u10*q0+u11*q1;
15    } } }
```

The CNOT gates are applied to two qubits: a control and a target qubit. If the control qubit is in state $|1\rangle$, the CNOT gate flips the target qubit. If the control qubit is in state $|0\rangle$, the CNOT gate does not change the target qubit. If we have two qubits and take the higher bit as the control qubit and the other as the target qubit, the CNOT gate is mathematically defined as

$$\text{CNOT} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix}. \quad (2)$$

When applied to the c -th and t -th qubits (control and target, respectively), the CNOT gate swaps the probability amplitude of $|x_n \cdots (x_c = 1) \cdots (x_t = 0) \cdots x_1\rangle$ with that of the $|x_n \cdots (x_c =$

$1) \cdots (x_t = 1) \cdots x_1$). The swaps, which affect half of the probability amplitudes of the quantum state, can also be performed in parallel. Listing 2 shows a pseudocode of the CNOT gate simulation.

Listing 2: Pseudocode of the CNOT gate simulation for the target q and the control qc qubits

```

1 void cnot(int q, int qc) {
2   if (q < qc) {
3     #pragma omp parallel for collapse(3)
4     for (int i=0; i<(1<<n); i+=(1<<(qc+1))) {
5       for (int j=(1<<qc); j<(1<<(qc+1)); j+=(1<<(q+1))) {
6         for (int k=0; k<(1<<(q)); k++) {
7           auto pos = i|j|k;
8           auto tmp = qreg[pos];
9           qreg[pos] = qreg[pos|(1<<q)];
10          qreg[pos|(1<<q)] = tmp;
11        } } } else {
12      #pragma omp parallel for collapse(3)
13      for (int i=0; i<(1<<n); i+=(1<<(q+1))) {
14        for (int j=0; j<(1<<(q)); j+=(1<<(qc+1))) {
15          for (int k=(1<<qc); k<(1<<(qc+1)); k++) {
16            auto pos = i|j|k;
17            auto tmp = qreg[pos];
18            qreg[pos] = qreg[pos|(1<<q)];
19            qreg[pos|(1<<q)] = tmp;
20          } } } }

```

2.2 Parallelization in distributed environment

The OpenMP program is used to efficiently parallelize the u3 and CNOT loops in a single machine by adding simple annotations, such as that on line 7 in Listing 1 and on lines 3 and 12 in Listing 2. However, in general, OpenMP does not work in distributed memory, such as memory of multiple nodes and GPUs. Therefore, simulators for distributed memory use message passing among machines and GPUs to parallelize the loops in addition to using OpenMP.

The QuEST and *qHipSTER* programs use similar strategies to parallelize the u3 and CNOT loops in multiple machines: use 2^m message passing interface (MPI) processes in the machines, partition $qreg$ among them equally, and parallelize them by exchanging probability amplitudes in $qreg$ through an MPI. Because a process manages 2^{n-m} probability amplitudes, u3 and CNOT are performed individually in each process if the number of (target) qubits is less than $n - m$; otherwise, probability amplitudes are exchanged. As shown in Listings 1 and 2, updates of $qreg[pos]$ and $qreg[pos + 2^q]$ are pairwise. Therefore, if a process p_i updates $qreg[pos]$, p_i receives $qreg[pos + 2^q]$ from another process $p_{i+2^q-(n-m)}$.

A naïve implementation with MPI processes, like that of QuEST, copies all the remote amplitudes into a local temporary buffer and then updates the local amplitudes, as shown in Figure 1 A. However, this approach requires additional memory space with a size the same as that of the local amplitudes. Therefore, in an MPI process, the local amplitudes are divided into small 2^l chunks as a computation unit to reduce the temporary buffer size, as shown in Figure 1 B. Once a remote chunk is copied, a local chunk is updated. In *qHipSTER*, as illustrated in Figure 1 C, the received probability amplitudes are also updated and returned to the sender. All the processes update the same number of chunks and balance the workloads among the processes. This approach may reduce communication overhead by dividing communication into two phases.

QuEST and *qHipSTER* use the MPI_SENDRECV function to exchange probability amplitudes, which is a blocking function. This communication does not overlap computation.

If GPUs accelerate computation in Listings 1 and 2, a kernel in a GPU runs as a process in Figure 1: probability amplitudes are exchanged among GPUs to enable fast qubit operations. However, this approach does not use GPU memory efficiently. In general, in addition to the user's GPU programs, the system components use GPU memory to run the programs. Even if a GPU has 32 GB, its program cannot use all of them: the program can use only 16 GB to store probability amplitudes if they are partitioned into 2^m processes.

3 PARTITIONING INTO CPUS AND GPUS

The probability amplitudes are stored in GPU memory in a cluster as much as possible. If there is not enough memory to store all the probability amplitudes, the remaining ones are stored in CPU memory. Each GPU and CPU is responsible for updating the local probability amplitudes during a simulation. Therefore, they must be exchanged among the GPUs and CPUs when simulating many qubits.

To simplify communication and synchronization between MPI processes, we limit the number of processes to the power of 2, which is similar to existing work as described in Subsection 2.2, and assume that all the processes can use the same number of GPUs. This limitation guarantees that all the MPI processes perform the same procedures at the same time. Note that every process has 2^{n-m} local probability amplitudes distributed by block decomposition, where n is the number of qubits to be simulated and m is the number of MPI processes.

3.1 Chunk allocation

Each process divides a local problem, which consists of 2^{n-m} probability amplitudes, into smaller problems called *chunks*, as shown in Figures 1 B and C. A chunk has 2^{n-m-l} probability amplitudes, where 2^l is the size of the chunks in a process.

Chunks are allocated first to the GPU memory, and then to the CPU memory. If all the chunks are in GPU memory, the computational power and memory bandwidth of the GPUs can fully accelerate QC simulation. We assume that a process has g GPUs and that a GPU can store c chunks, which contain $2^{n-m-l} \times c$ amplitudes in total. Therefore, if 2^n is smaller or equal to $2^{n-m-l} \times c \times g \times m$, all the amplitudes are stored in GPU memory; otherwise, some spill over into CPU memory.

In addition, each GPU has a buffer for communicating with the other GPUs and CPUs. Each CPU also has a buffer for communicating with CPUs in other processes, and with GPUs in the same and other processes. We overlap computation and data transfer by pipelining them, so each buffer has enough space for processing two or more chunks.

Figure 2 shows memory allocation for an MPI process p_i that uses two GPUs ($g = 2$) and divides local amplitudes into eight chunks ($l = 3$). The first GPU manages the first three chunks ($c = 3$), which contain amplitudes from $[2^{n-m} \times i]$ to $[2^{n-m} \times i + 2^{n-m-3} \times 3 - 1]$. The second GPU manages the next three chunks ($[2^{n-m} \times i + 2^{n-m-3} \times 3]$ to $[2^{n-m} \times i + 2^{n-m-3} \times 6 - 1]$). The

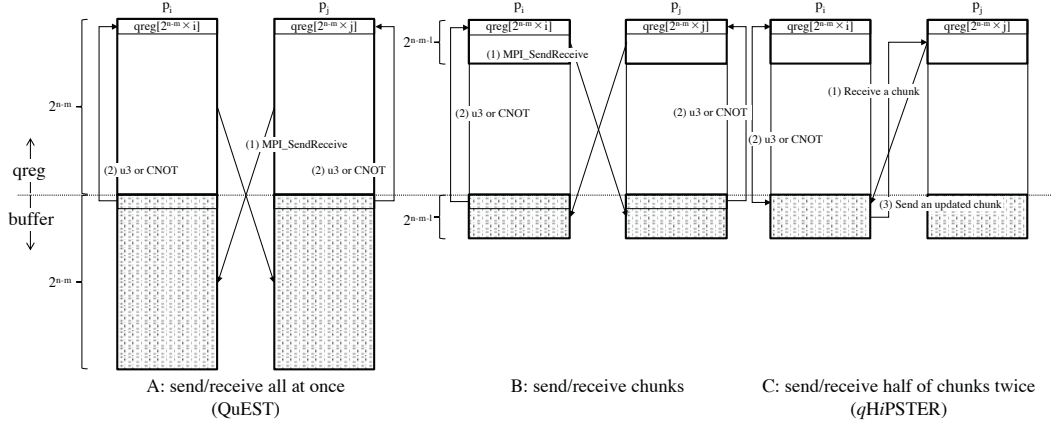


Figure 1: Temporary buffer used to store copies of probability amplitudes in another MPI process. Total storage can be reduced by dividing problem into small chunks (B and C). q HiPSTER divides communication into two phases (C).

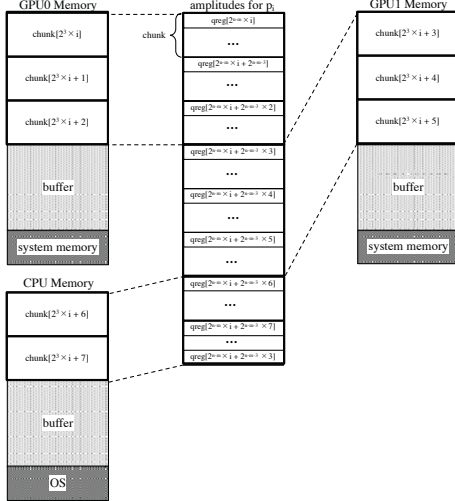


Figure 2: Memory allocation for MPI process p_i , which uses two GPUs ($g = 2$) and divides local amplitudes into eight chunks ($l = 3$). Each GPU memory stores three chunks ($c = 3$).

CPU manages the remaining chunks ($[2^{n-m} \times i + 2^{n-m-3} \times 6]$ to $[2^{n-m} \times i + 2^{n-m-3} \times 8 - 1]$, which is $[2^{n-m} \times (i + 1) - 1]$). Each GPU memory and CPU memory has its own buffer.

The chunk size (2^{n-m-l}) must be configured carefully to fill each GPU memory (the smaller the chunks the better) and maximize the efficiency of GPU calculation and data transfer (the larger the better). In our experience, at most 15.5 GB out of 16 GB are available for the user program on a GPU, so the largest chunk size to fill 15.5 GB with chunks and temporary buffers is 512 MB. With this size ($l = 25$), 30 chunks and 1 temporary buffer can be stored on a GPU. Pipelining to overlap computation and data transfer further optimizes the operation. This pipelining requires two or

more temporary buffers in each GPU to process two or more chunks in parallel (discussed below).

3.2 Chunk exchange

Two pairwise calculations are required for u3 and CNOT, as shown in Listings 1 and 2. The distance between probability amplitude pairs is determined on the basis of the operated qubit; it is 2^q for a gate on q qubit. Therefore, depending on values of q , n , m , l , and c , some chunks must be transferred.

A process can transfer chunks in one of four ways, as shown in Figure 3.

- Intra-process CPU-GPU: A chunk in CPU memory is copied to the buffer of a GPU memory within the same process.
- Intra-process GPU-GPU: A chunk in GPU memory is copied to the buffer of another GPU memory within the same process.
- Inter-process CPU-CPU: A chunk in CPU memory is copied to the buffer of a CPU memory in another process.
- Inter-process GPU-GPU: A chunk in GPU memory is copied to the buffer of a GPU memory in another process.

The chunks are processed, and when necessary moved in a machine configuration, as illustrated in Figure 3.

4 HETEROGENEOUS PARALLELIZATION

We exploit hybrid parallelization for gate simulation in a quantum circuit by using MPI to parallelize operations on the cluster, OpenMP to parallelize kernels on the host computer cores, and CUDA [6] to parallelize kernels on the GPUs. Probability amplitudes are stored in GPU and CPU memory in the cluster, as described in Section 3. Each GPU and CPU is responsible for updating local probability amplitudes.

4.1 u3 gate

Our simulator does not calculate all the computation for the u3 gates in the GPUs. A kernel to simulate a u3 gate of q is to load two amplitudes using pairwise operation, multiply them by a matrix

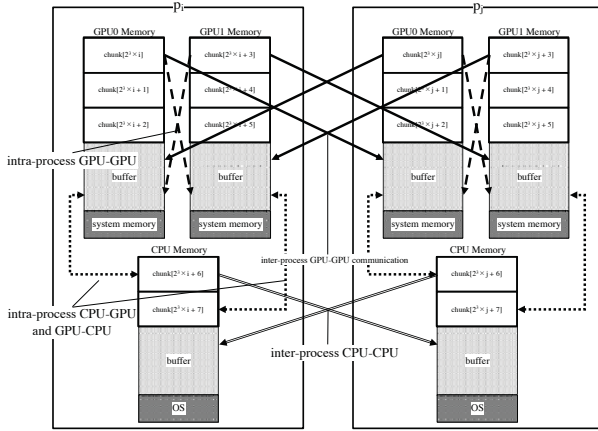


Figure 3: Chunk exchange between two MPI processes (p_i and p_j); each has two GPUs ($g = 2$), and local probability amplitudes are divided into eight ($l = 3$) chunks.

defined by three parameters (Eq. (1)), and store the product. Multiplying a matrix takes 28 flops, which is less than what needed for loading and storing 32 bytes of two complex numbers of amplitudes (the byte/flop ratio is about 2.29) into local and remote memory. If the GPUs calculated all the computation, chunk exchanges among CPUs and GPUs would be a bottleneck.

Our simulator changes strategies to simulate $u3$ gates depending on the q , n (the qubit of a circuit), m (the number of processes), 2^l (the number of chunks in a process), g (the number of GPUs in a process), and c (the number of chunks in GPU memory).

4.1.1 in-chunk. If q is smaller than 2^{n-m-l} , each chunk is independently calculated without any chunk exchanges. The CPUs and GPUs process the pseudo code for a $u3$ gate in Listing 1 using OpenMP and CUDA parallelization.

4.1.2 in-process. If 2^q is larger or equal to 2^{n-m-l} and smaller than 2^{n-m} , two chunks are a pair in an MPI process. If both are in the memory of a CPU or a GPU, the simulator updates them using the code in Listing 1 and OpenMP parallelization; otherwise, it updates them by exchanging chunks.

If two chunks are in the memories of different GPUs ($(c \times 2^{n-m-l}) \leq 2^q < c \times g \times 2^{n-m-l}$ and the larger index of two amplitudes is less than $2^m \times i + c \times g \times 2^{n-m-l}$ in process p_i), the simulator moves both chunks to different GPUs, as illustrated in Figure 1 B, and each GPU updates the local amplitudes.

If one chunk is in CPU memory and the other chunk is in GPU memory (the smaller index of the two amplitudes is less than $2^m \times i + c \times g \times 2^{n-m-l}$), the chunk in CPU memory is transferred to GPU memory, the GPU updates the amplitude in the chunk, and then the updated chunk is returned to CPU memory. This is similar to the operation illustrated in Figure 1 C.

4.1.3 inter-processes. If 2^q is larger or equal to 2^{n-m} , two chunks are a pair in two MPI processes. We assume that every process has the same configuration of GPUs and memory allocation. Therefore,

the two chunks are in GPU memory or in CPU memory. The simulator moves both chunks to different GPUs, as illustrated in Figure 1 B, and each CPU and GPU updates the local probability amplitudes.

4.1.4 pipelining. Chunk exchange between CPUs and GPUs has a large overhead compared to the kernel calculation itself, so we overlap the exchange and kernel calculations using a pipelining technique. We exploit multiple CUDA streams for probability amplitude exchange and kernel computation: while a chunk is exchanged in one stream, probability amplitudes in the other chunk are updated in the other stream. Before processing on the CPU, all chunks on the GPUs are queued in the CUDA streams, enabling the CPU cores to execute kernel calculations in parallel. After the CPU cores calculate all the chunks, they are synchronized with the CUDA streams.

4.1.5 $u1$ gate. We implement special cases of the $u3$ gate kernel as a separate kernel to eliminate unnecessary calculation and data exchange. The $u1$ gate, which is an important gate in quantum circuits, is the special case of a $u3$ gate kernel with zero values for parameters θ and ψ , so the gate matrix is simplified as

$$u1(\lambda) = u3(0, 0, \lambda) = \begin{pmatrix} 1 & 0 \\ 0 & e^{i\lambda} \end{pmatrix}. \quad (3)$$

As the matrix of $u1$ is diagonal, the simulator updates a pair of amplitudes independently without them referring to each other. Therefore, chunk exchanges are unnecessary. In addition, because only the second probability amplitude is updated in the $u1$ gate kernel, we can halve the memory access. The byte/flop ratio of a $u1$ gate kernel per amplitude is 5.33, which is more than double that of a $u3$ gate. Memory stall occurs more often in a $u1$ gate kernel than in a $u3$ gate kernel.

4.2 CNOT gate

A kernel to simulate a CNOT gate of q as a target qubit and q_c as a control qubit is a pairwise operation to load, swap, and store two probability amplitudes. Similar to simulation of $u3$ gates, the simulator changes strategies to simulate CNOT gates on the basis of n , m , 2^l , g , and c , in addition to q and q_c .

4.2.1 in-chunk. If q is smaller than 2^{n-m-l} , each chunk is independently calculated without any chunk exchanges. The CPUs and GPUs process the code in Listing 2 using OpenMP parallelization.

4.2.2 in-process. If 2^q is larger or equal to 2^{n-m-l} and smaller than 2^{n-m} , two chunks are a pair in an MPI process. If 2^q and 2^{q_c} are larger or the same as 2^{n-m-l} , all probability amplitudes in a chunk are exchanged between the chunk pairs. In this case, the simulator does not exchange any chunks. Instead, it maintains an index table that indicates where the latest probability amplitudes of a chunk exist in memory.

If a CNOT gate exchanges two chunks ($\text{chunk}[i]$ and $\text{chunk}[j]$), the simulator maintains an index table indicating that the actual i chunk is in $\text{chunk}[j]$ and vice versa by mapping from i to j and j to i . In the gate simulation described below, if $\text{chunk}[i]$ is required, $\text{chunk}[j]$ is accessed.

If 2^{q_c} is smaller than 2^{n-m-l} , half of probability amplitudes in a chunk are swapped with half in the pair chunk. Exchanges are

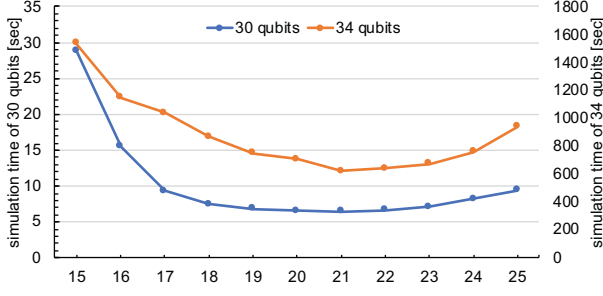


Figure 4: Simulation times against the number of probabilistic amplitudes in a chunk for two nodes simulating QFT circuits. A 30-qubit QFT can be computed on GPUs, while a 34-qubit QFT must be computed heterogeneously.

done in the same manner as for a kernel in a u3 gate; the simulator works as shown in Listing 2 if both are in the memory of a CPU or a GPU, as shown in Figure 1 B if they are in the memory of different GPUs, and as shown in Figure 1-C if they are in the memory of a CPU and a GPU.

4.2.3 inter-processes. If 2^q is larger or equal to 2^{n-m} , two chunks are a pair in two MPI processes. As with the in-process procedure, if 2^{q_c} is larger or the same size as 2^{n-m-l} , the simulator avoids actual data communication between the MPI processes. Otherwise, it processes a CNOT gate as with the inter-processes procedure for a u3 gate.

5 PERFORMANCE EVALUATION

To evaluate our simulator, which is written in C++ and parallelized using MPI, OpenMP, and CUDA, we configured a cluster of IBM AC922 Power System [3] that consists of two IBM POWER9 sockets with a 3.07 GHz, 22 cores, and 4 simultaneous multi-threading, 512 GB memory, and 6 NVIDIA Tesla V100 GPUs. We used the CUDA 9.1, IBM Spectrum MPI 10.1.0, and the IBM XLC/C++ 13.1.6 compiler on the Red Hat Enterprise Linux 7.3 OS. Though all nodes in a cluster and GPUs in a node are connected via InfiniBand EDR and a NVLink2 link [7], respectively, our implementation is portable to any environment because we do not use any other their unique features.

5.1 Unit size consideration

We first found the best chunk size (2^{n-m-l}) by simulating a quantum Fourier transform (QFT) circuit, a well-known component for many quantum computing applications. It is composed of u3, u1, and CNOT gates. We used a QFT circuit on 30 qubits with all the probability amplitudes stored in GPU memory, and on 34 qubits with some probability amplitudes stored on CPU memory.

Figure 4 compares the simulation times for QFT circuits using two nodes of the OpenPOWER cluster. In general, there is a tradeoff in chunk size: a small chunk size reduces the number of chunk exchanges in the CNOT kernel and thus reduces memory usage in the host CPU which a large chunk size improves the efficiency of data transfer and kernel execution on GPUs. From these results, we

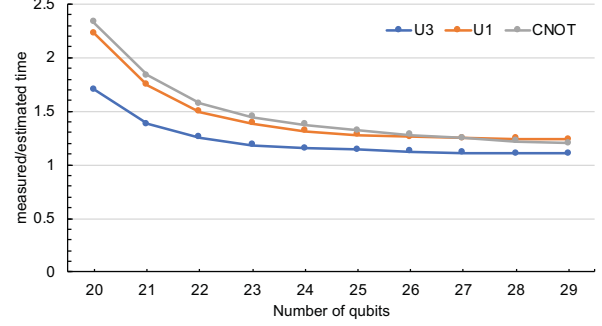


Figure 5: Ratio of measured and estimated runtimes for u3, u1, and CNOT gate kernels on single Tesla V100 GPU. Measured roofline performance was almost 90%, 80%, and 80% efficiency, respectively.

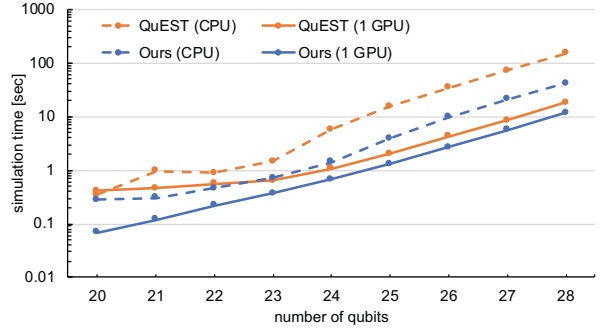


Figure 6: Comparison of QFT simulation time with QuEST and our simulator running on a CPU only and on a CPU with a single Tesla V100 GPU.

determined that 2^{21} is the best chunk size to balance this tradeoff on the OpenPOWER cluster.

5.2 Single-GPU performance

Next, we evaluated the efficiency of our implementation by using the roofline model [21]. We enabled a single GPU and configured a QFT circuit to use only GPU memory. We compared the measured runtimes of the u3, u1, and CNOT gates in the circuit with the estimated runtimes, which were calculated using byte/flop ratios of 2.29 (u3) and 5.33 (U1). The CNOT gate does not required any computation, so we used the estimation of the memory bandwidth as the runtime.

Figure 5 shows the measured and estimated time ratios for the u3, u1, and CNOT gate kernels. Good efficiency compared to the estimation was observed when number of qubits was sufficiently large; almost 90% roofline efficiency we achieved for the u3 gate.

We also compared the simulation time of our simulator with that of the QuEST program [12], which is an open source QC simulator that supports distributed and single-GPU environments (multi-GPU environments are not yet supported). Figure 6 compares the performance of our simulator with that of QuEST while running QFT simulation in two environments: on a CPU only and on a CPU

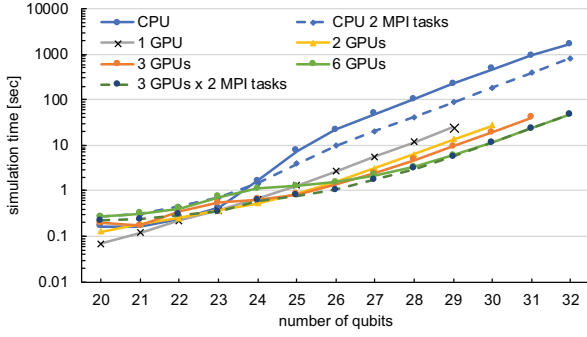


Figure 7: Single-node QFT simulation time for different combinations, with or without GPUs, with different numbers of GPUs, and with or without MPI.

with a single Tesla V100 GPU. The runtime with our simulator was about 2/3 less than that with QuEST in both environments.

Figure 6 also shows that our simulator always improved performance of 20 or more qubits simulation with a GPU. In all of the evaluation with a GPU, only GPU memory stores chunks. A GPU reduced at least 50 % time of simulation on CPU.

5.3 Single-node performance

Our QC simulator supports computers with or without GPUs. We measured the contribution of GPU acceleration in heterogeneous parallelization by using QFT circuits on a single node. We evaluated the single-node performance in various combinations: with or without GPUs, different numbers of GPUs, and with or without MPI processes.

Figure 7 shows plots the results of QFT simulation on a single node. A comparison of the number of GPUs shows that the simulation time scales with the number of GPUs. In all of the evaluation with GPUs, only GPU memory stores chunks. For simulation of large qubits (such as 28 and 29), performance was improved based on the number of GPUs. For example, in simulation of 29 qubits, two, three, and six GPUs reduced simulation time of single-GPU by 47 %, 62 %, and 75%, respectively. On the other hand, for simulation of small qubits (such as 20 and 21), overhead due to parallelization reduced performance. Two MPI tasks showed better results than single MPI task for both with a CPU only and with six GPUs. In NUMA and multi-GPU environment, access to memory in a remote NUMA node and GPU is expensive. In our evaluation, we mapped each task to each POWER9 socket. With this configuration, memory affinity and the connection of NVLink between the CPU and GPUs are optimized and then access to remote memory is reduced.

Figure 8 shows the simulation times of our simulator with and without GPUs, and Figure 9 shows the ratio of memory allocation of chunks on the CPU against the total number of qubits and the ratio of simulation time reduction due to using GPUs. The CPU memory ratio significantly increased from 33 qubits while the simulation time of the heterogeneous run also increased. When the number of qubits was 33 and 34, the processing times of chunks on GPUs were almost hidden behind the processing times of chunks on the

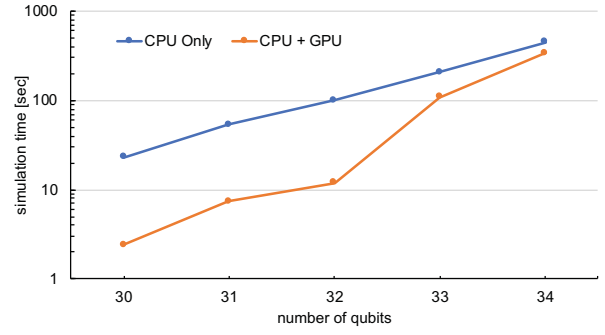


Figure 8: Simulation times of CPU-only implementation and CPU + GPU heterogeneous implementation on single node. 35-qubit simulation, which requires 512 GB of memory, can only be run with heterogeneous implementation.

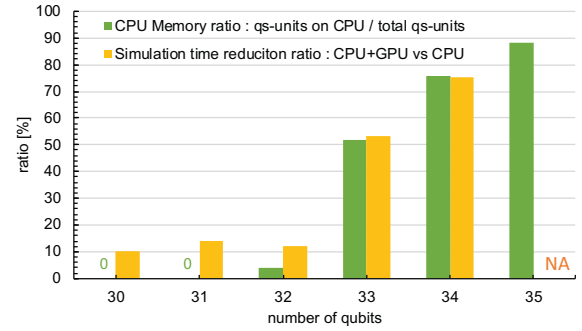


Figure 9: Ratio of memory usage on CPU and ratio of time reduction with heterogeneous implementation. For 33- and 34-qubit simulations, the two ratios were similar, suggesting that GPU computation was hidden behind CPU computation. Simulation time ratio was not available for 35-qubit simulation.

CPU as the CPU memory ratios and the simulation time ratios were almost the same for those numbers of qubits.

In addition, the substantial improvement achieved with our heterogeneous implementation enabled us to simulate a 35-qubit QFT circuit on a single node with only 512 GB of memory space required to store the probability amplitudes. With only a CPU, we could simulate up to 34-qubit circuits. Simulating 35-qubit circuits is not possible because we cannot allocate 512-GB arrays on a node that has 512 GB of total memory because the OS kernels, drivers and other arrays in the program are already using part of the memory. Instead, we allocated 84-GB arrays for the probability amplitudes on the six GPUs so that we could fit the remaining chunks in CPU memory. This enabled us to add one more qubit to be simulated with the given resources.

5.4 Scalability

We evaluated our heterogeneous QC simulator on the OpenPOWER cluster by using a QFT circuit, and we prepared quantum circuits with various numbers of qubits to examine its scalability.

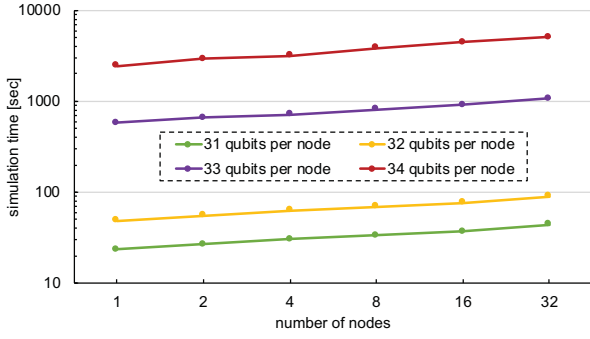


Figure 10: Weak scaling of QFT simulation time measured on OpenPOWER cluster using our QC simulator up to 32 nodes.

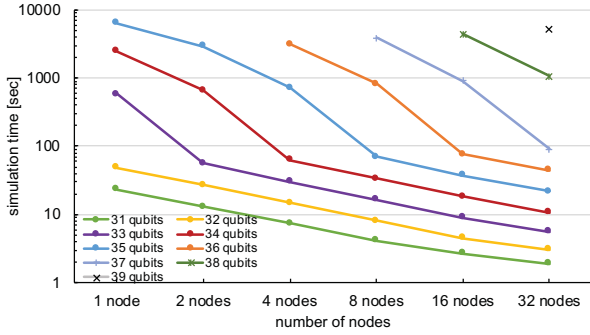


Figure 11: Strong scaling of QFT simulation time measured on OpenPOWER cluster using our QC simulator up to 32 nodes.

Figure 10 shows the weak scaling of the QFT circuit simulation on our QC simulator for up to 32 nodes. We fixed the number of qubits per node so as to observe the weak scalability. All probability amplitudes were stored in GPU memory for 31 and 32 qubits per node. The memory allocation ratio on the CPU for all probability amplitudes was 34% for 33 qubits per node and 67% for 34 qubits per node (as determined by referring to Figure 9). In all cases, we observed MPI communication overheads that increased the simulation time when we increased the number of nodes.

Figure 11 shows the strong scaling of the QFT circuit simulation on our heterogeneous QC simulator for up to 32 nodes. We ran 31 to 39 qubits QFT and observed super scaling when the size of the amplitude array fit into GPU memory and observed linear scaling in the other cases. We could add one more qubit only to the 1-node case for the QFT simulation. For multiple nodes, the simulation was unstable due to the lack of memory space (this is why, in Figure 11, up to the 35-qubit, the result for the 1-node case is shown but not for 36-qubit or larger on multiple nodes). We presume this is because an MPI process uses a large memory space for optimization, and the instability is due to the failure to allocate memory inside the process. For a stable run, we need a lighter weight OS kernel, or an improved MPI implementation to utilize free memory space,

or use more GPUs per node, as well as use 32 GB version of the Tesla V100 GPU to increase the free memory space.

6 CONCLUSION

Our proposed heterogeneous approach to simulating QC on a classical computer is to efficiently accelerate and parallelize QC simulation using both a CPU core and GPUs. To track the evolution of the quantum state of n qubits (represented as a 2^n dimensional complex vector of probability amplitudes), we hierarchically decompose the probability amplitudes into units called chunks. We distribute the chunks to both CPU and GPU memory and design the computation in a unit-wise manner so that we can utilize the total memory space as efficiently as possible by reducing the size of the buffer required for communication between the CPU and GPUs and between MPI tasks. We use optimized quantum gate kernels processed in each chunk and overlap computation and data exchange to mitigate the communication latency. We use a specialized kernel for the u1 gate to avoid data exchange between chunks and between MPI tasks, where the u1 gate is a special case of the u3 gate. For the CNOT gate kernel, we use a chunk index table for data exchange between chunks and MPI tasks by swapping table indices without actual data transfer.

We evaluated our heterogeneous quantum computing simulator with quantum Fourier transform circuits on an OpenPOWER cluster, which had GPUs connected via NVLink. We observed 10+ times speed-up due to the GPUs when most of the amplitudes fit in GPU memory. We also observed strong scaling for up to 32 nodes when we simulated quantum circuits with up to 39 qubits.

Our heterogeneous implementation enables memory to be utilized efficiently. Even if the storage required for 2^n probability amplitudes does not fully fit in the given CPU memory (for example, when the size is the power of 2 and there is some overhead due to OS and/or MPI processing), all probability amplitudes can still be stored by distributing the chunks to both GPU and CPU memory.

Though a size of single-GPU memory is recently increasing, a size of CPU memory is also increasing. We estimate that our heterogeneous implementation will be effective on the future computer architecture. Noise of quantum computing is simulated with unitary and kraus noise models that require similar calculation with gate simulation [18]. We plan to integrate our simulator with qiskit-aer to support their noise models.

For further optimization, we will not only implement more specialized gate kernels to avoid data transfer between chunks and MPI tasks but also fuse sequent gate kernels to block amplitudes on the local memory. Currently, we do not offload computation from CPU to GPU if GPUs are fully allocated because the bandwidth of NVLink is smaller than the memory bandwidth on the CPU. If a faster link that exceeds the memory bandwidth appears in the future, we will implement computation offloading to accelerate more. Finally, we will evaluate additional real quantum computing applications and increase the number of qubits to 50 or more as part of our contribution to the quantum computer development.

ACKNOWLEDGMENTS

The authors would like to thank Christopher J. Wood, John A. Smolin and Jay Gambetta for base implementation and useful discussions for our quantum computing simulator.

REFERENCES

- [1] Alán Aspuru-Guzik, Anthony D Dutoi, Peter J Love, and Martin Head-Gordon. 2005. Simulated quantum computation of molecular energies. *Science* 309, 5741 (2005), 1704–1707.
- [2] Sergio Boixo, Sergei V Isakov, Vadim N Smelyanskiy, and Hartmut Neven. 2017. Simulation of low-depth quantum circuits as complex undirected graphical models. *arXiv preprint arXiv:1712.05384* (2017).
- [3] AB Caldeira. 2018. IBM power system AC922 introduction and technical overview. *IBM Redbooks* (2018).
- [4] Zhao-Yun Chen, Qi Zhou, Cheng Xue, Xia Yang, Guang-Can Guo, and Guo-Ping Guo. 2018. 64-qubit quantum circuit simulation. *Science Bulletin* (2018).
- [5] Microsoft Corporation. 2018. The Q# Programming Language. Retrieved January 11, 2019 from <https://www.microsoft.com/en-us/quantum/development-kit>
- [6] NVIDIA Corporation. 2007. CUDA Toolkit. Retrieved January 11, 2019 from <https://developer.nvidia.com/cuda-toolkit>
- [7] NVIDIA Corporation. 2016. NVLink Interconnect Technology. Retrieved January 11, 2019 from <http://www.nvidia.com/object/nvlink.html>
- [8] NVIDIA Corporation. 2018. NVIDIA Tesla V100. Retrieved January 11, 2019 from <https://www.nvidia.com/en-us/data-center/tesla-v100/>
- [9] Andrew W Cross, Lev S Bishop, John A Smolin, and Jay M Gambetta. 2017. Open quantum assembly language. *arXiv preprint arXiv:1707.03429* (2017).
- [10] Eladio Gutierrez, Sergio Romero, Maria A Trenas, and Emilio L Zapata. 2008. Parallel quantum computer simulation on the CUDA architecture. In *International Conference on Computational Science*. Springer, 700–709.
- [11] Thomas Häner and Damian S Steiger. 2017. 0.5 petabyte simulation of a 45-qubit quantum circuit. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*. ACM, 33.
- [12] Tyson Jones, Anna Brown, Ian Bush, and Simon Benjamin. 2018. QuEST and High Performance Simulation of Quantum Computers. *arXiv preprint arXiv:1802.08032* (2018).
- [13] Abhinav Kandala, Antonio Mezzacapo, Kristan Temme, Maika Takita, Markus Brink, Jerry M Chow, and Jay M Gambetta. 2017. Hardware-efficient variational quantum eigensolver for small molecules and quantum magnets. *Nature* 549, 7671 (2017), 242.
- [14] Zhen Li and Jiabin Yuan. 2017. Quantum Computer Simulation on GPU Cluster Incorporating Data Locality. In *International Conference on Cloud Computing and Security*. Springer, 85–97.
- [15] PJJ O'malley, Ryan Babbush, Ian D Kivlichan, Jonathan Romero, Jarrod R McClean, Rami Barends, Julian Kelly, Pedram Roushan, Andrew Tranter, Nan Ding, et al. 2016. Scalable quantum simulation of molecular energies. *Physical Review X* 6, 3 (2016), 031007.
- [16] Edwin Pednault, John A Gunnels, Giacomo Nannicini, Lior Horeh, Thomas Magerlein, Edgar Solomonik, and Robert Wisnieff. 2017. Breaking the 49-qubit barrier in the simulation of quantum circuits. *arXiv preprint arXiv:1710.05867* (2017).
- [17] Qiskit. 2017. Qiskit: An Open-source Framework for Quantum Computing. Retrieved January 11, 2019 from <https://qiskit.org/>
- [18] Qiskit. 2019. Qiskit Aer: A High Performance Simulator Framework for Quantum Circuits. Retrieved January 11, 2019 from <https://qiskit.org/aer>
- [19] Mikhail Smelyanskiy, Nicolas PD Sawaya, and Alán Aspuru-Guzik. 2016. qHiP-STER: the quantum high performance software testing environment. *arXiv preprint arXiv:1601.07195* (2016).
- [20] Damian S. Steiger, Thomas Häner, and Matthias Troyer. 2016. ProjectQ: An Open Source Software Framework for Quantum Computing. *Quantum* 2, 49 (2018). <https://doi.org/10.22331/q-2018-01-31-49> arXiv:arXiv:1612.08091
- [21] Samuel Williams, Andrew Waterman, and David Patterson. 2009. Roofline: an insightful visual performance model for multicore architectures. *Commun. ACM* 52, 4 (2009), 65–76.
- [22] Pei Zhang, Jiabin Yuan, and Xiangwen Lu. 2015. Quantum computer simulation on Multi-GPU incorporating data locality. In *International Conference on Algorithms and Architectures for Parallel Processing*. Springer, 241–256.